

Sistema de Locadora de Veículos: Modelagem de Dados e Backend RESTful com ASP.NET Core e Entity Framework

Cecília [Sobrenome]¹

Curso de [Disciplina] — Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte — MG — Brasil seu.email@sga.pucminas.br

Resumo

Este relatório descreve o desenvolvimento de um sistema de gerenciamento para uma locadora de veículos, contemplando a modelagem do banco de dados relacional, a implementação de uma API RESTful em C# com ASP.NET Core e Entity Framework Core, e a documentação via Swagger. O sistema implementa seis entidades inter-relacionadas, operações CRUD completas, validação de dados e cinco rotas de filtro que empregam diferentes tipos de junção entre tabelas. Discutem-se as decisões de projeto, a arquitetura adotada e os testes realizados.

Abstract

This report describes the development of a management system for a vehicle rental company, covering relational database modeling, the implementation of a RESTful API in C# with ASP.NET Core and Entity Framework Core, and Swagger documentation. The system implements six interrelated entities, full CRUD operations, data validation, and five filter routes employing different types of table joins. Design decisions, the adopted architecture, and the tests performed are discussed.

1. Introdução

A gestão de uma locadora de veículos envolve o controle simultâneo de frota, clientes, contratos de aluguel e pagamentos. Realizar esse controle de forma manual ou por planilhas é propenso a erros de consistência — como alugar um veículo já comprometido ou perder o histórico de devoluções. Um sistema informatizado apoiado por um banco de dados relacional resolve esses problemas ao centralizar a informação e impor regras de integridade.

Este trabalho tem como objetivo desenvolver um sistema completo de *backend* para uma locadora de veículos, capaz de: (i) cadastrar e gerenciar veículos, fabricantes,

categorias e clientes; (ii) registrar aluguéis e suas devoluções, calculando valores automaticamente; (iii) processar pagamentos com controle de saldo; e (iv) oferecer consultas analíticas por meio de filtros que cruzam múltiplas tabelas.

As tecnologias empregadas foram C# com ASP.NET Core para a construção da API, Entity Framework Core como mapeador objeto-relacional (ORM), SQL Server Express como sistema gerenciador de banco de dados (SGBD) e Swagger (OpenAPI) para documentação e teste interativo das rotas.

2. Requisitos Funcionais

O sistema atende aos seguintes requisitos funcionais:

- **RF01 — Gestão de fabricantes:** cadastrar, consultar, editar e excluir fabricantes (marcas), com bloqueio de exclusão quando há veículos vinculados.
- **RF02 — Gestão de categorias:** manter categorias de veículos (econômico, SUV, luxo etc.), cada uma com um fator multiplicador de preço.
- **RF03 — Gestão de veículos:** registrar veículos vinculados obrigatoriamente a um fabricante e a uma categoria, com modelo, ano de fabricação, quilometragem, placa e valor de diária.
- **RF04 — Gestão de clientes:** cadastrar clientes com nome, CPF (único), e-mail (único) e saldo em carteira.
- **RF05 — Registro de aluguéis:** criar um aluguel associando cliente e veículo em um período, calculando automaticamente o valor da diária (diária base $\times$ fator da categoria) e o valor total.
- **RF06 — Devolução:** registrar a devolução do veículo, gravando data e quilometragem finais, recalculando o valor em caso de atraso e liberando o veículo.
- **RF07 — Recarga e pagamento:** recarregar o saldo do cliente e processar pagamentos, debitando a carteira quando a forma escolhida é "saldo".
- **RF08 — Consultas analíticas:** oferecer filtros que cruzam tabelas, como faturamento por fabricante e resumo de aluguéis por cliente.

2.1. Casos de Uso Principais

O caso de uso central é o *Realizar Aluguel*: o atendente seleciona um cliente e um veículo disponível, define o período e o sistema calcula o custo, marca o veículo como indisponível e cria o registro. Seu complemento é o *Registrar Devolução*, que encerra o contrato, atualiza a quilometragem do veículo e o disponibiliza novamente.

3. Arquitetura do Sistema

A aplicação segue uma arquitetura em camadas inspirada no padrão MVC, adaptada para uma API sem camada de visão renderizada no servidor (a visão é o cliente *front-end* consumidor). As responsabilidades dividem-se em:

- **Camada de Modelo (Models):** classes de entidade que representam as tabelas e seus relacionamentos, anotadas com regras de validação.
- **Camada de Acesso a Dados (Data):** a classe `AppDbContext`, que configura o mapeamento objeto-relacional, restrições de integridade e a carga inicial de dados (*seed*).
- **Camada de Controle (Controllers):** controladores que expõem os *endpoints* REST, orquestram a lógica de negócio e retornam respostas HTTP apropriadas.
- **Objetos de Transferência (DTOs):** estruturas que isolam o contrato da API do modelo interno, evitando exposição direta das entidades e ciclos de serialização.

Essa separação favorece a manutenibilidade: alterações no esquema do banco ficam contidas na camada de modelo e dados, enquanto o contrato externo da API permanece estável através dos DTOs.

4. Implementação Técnica

4.1. Tecnologias

O *backend* foi construído sobre o ASP.NET Core (`net8.0`), utilizando os pacotes `Microsoft.EntityFrameworkCore` para persistência e `Swashbuckle.AspNetCore` para a documentação OpenAPI. O EF Core opera no modo *Code First*: as tabelas são derivadas das classes C#.

4.2. Principais Classes

A classe `AppDbContext` concentra a configuração do modelo. O trecho a seguir ilustra a definição de índices únicos e do comportamento de exclusão, que evita deleções em cascata indesejadas:

```
1 modelBuilder.Entity<Cliente>()  
2     .HasIndex(c => c.Cpf).IsUnique();  
3  
4 modelBuilder.Entity<Aluguel>()  
5     .HasOne(a => a.Veiculo)  
6     .WithMany(v => v.Alugueis)  
7     .HasForeignKey(a => a.VeiculoId)  
8     .OnDelete(DeleteBehavior.Restrict);
```

Listing 1. Configuração de restrições no `AppDbContext`.

A lógica de negócio mais relevante está no `AlugueisController`, que calcula o valor do contrato a partir do fator de preço da categoria:

```
1 var fator = veiculo.CategoriaVeiculo?.FatorPreco ?? 1.0m;  
2 var valorDiaria = Math.Round(veiculo.ValorDiariaBase * fator, 2);  
3 var dias = (input.DataDevolucaoPrevista.Date  
4             - input.DataRetirada.Date).Days;  
5 if (dias < 1) dias = 1;  
6 var valorTotal = valorDiaria * dias;
```

Listing 2. Cálculo do valor do aluguel.

4.3. Escolhas de Projeto

Optou-se por `DeleteBehavior.Restrict` em vez de cascata para preservar o histórico: um cliente ou veículo com aluguéis não pode ser apagado, evitando perda de registros financeiros. A validação ocorre em duas frentes: anotações de dados (`DataAnnotations`) nos DTOs, verificadas automaticamente pelo *model binding*, e validações de regra de negócio nos controladores (ex.: veículo indisponível, saldo insuficiente).

5. Descrição do Banco de Dados

5.1. Modelo de Dados

O banco possui seis tabelas. A Tabela 1 resume as entidades e seus principais atributos.

Table 1. Entidades do sistema.

Entidade	Principais atributos
Fabricante	Id, Nome, PaisOrigem
CategoriaVeiculo	Id, Nome, Descricao, FatorPreco
Veiculo	Id, Modelo, AnoFabricacao, Quilometragem, Placa, ValorDiariaBase, Disponivel, FabricanteId (FK), CategoriaVeiculoId (FK)
Cliente	Id, Nome, Cpf (único), Email (único), Telefone, Cidade, Saldo
Aluguel	Id, ClienteId (FK), VeiculoId (FK), DataRetirada, DataDevolucaoPrevista, DataDevolucaoReal, QuilometragemInicial, QuilometragemFinal, ValorDiaria, ValorTotal, Devolvido
Pagamento	Id, AluguelId (FK), Valor, Forma, DataPagamento, Confirmado

5.2. Relacionamentos e Integridade

Os relacionamentos são: Fabricante 1:N Veículo; CategoriaVeiculo 1:N Veículo; Cliente 1:N Aluguel; Veículo 1:N Aluguel; e Aluguel 1:N Pagamento. As chaves estrangeiras garantem a integridade referencial, e índices únicos sobre CPF, e-mail e placa impedem duplicidades.

5.3. Consultas SQL Relevantes

O filtro de faturamento por fabricante, gerado pelo EF Core, corresponde aproximadamente à seguinte consulta SQL com junção e agrupamento:

```
1 SELECT f.Nome, COUNT(*) AS QtdAlugueis,
2      SUM(a.ValorTotal) AS Faturamento
3 FROM Alugueis a
4 INNER JOIN Veiculos v ON a.VeiculoId = v.Id
5 INNER JOIN Fabricantes f ON v.FabricanteId = f.Id
6 GROUP BY f.Nome
7 ORDER BY Faturamento DESC;
```

Listing 3. Faturamento por fabricante (INNER JOIN + GROUP BY).

6. Testes e Validação

Os testes foram conduzidos manualmente através da interface do Swagger, exercitando cada *endpoint*. Para cada entidade verificou-se o ciclo completo: criação (POST), listagem e busca (GET), atualização (PUT) e exclusão (DELETE). Validaram-se também os cenários de erro: criação de cliente com CPF duplicado (retorno 409), aluguel de veículo indisponível (409), pagamento com saldo insuficiente (400) e busca de registro inexistente (404).

Os cinco filtros com junções foram testados com os dados de carga inicial, confirmando que os resultados de faturamento, contagem e resumos correspondiam aos valores esperados. Não foram identificados problemas de consistência; o principal ajuste durante o desenvolvimento foi a inclusão de `ReferenceHandler.IgnoreCycles` na serialização JSON, para evitar laços infinitos entre entidades relacionadas.

7. Resultados e Conclusões

O sistema desenvolvido atende integralmente aos requisitos propostos: seis entidades (uma além do mínimo exigido), operações CRUD completas, cinco filtros com pelo menos dois tipos de junção (INNER JOIN e LEFT JOIN) e documentação interativa via Swagger. A modelagem relacional mostrou-se adequada para representar as regras do domínio, e o uso do Entity Framework no modo *Code First* agilizou a evolução do esquema.

Conclui-se que a solução é eficaz para as necessidades de uma locadora de veículos de pequeno e médio porte, oferecendo controle confiável da frota, dos contratos e das

finanças.

8. Considerações Finais

O desenvolvimento consolidou conceitos de modelagem de dados, mapeamento objeto-relacional e construção de APIs REST. Como melhorias futuras, sugerem-se: autenticação e autorização de usuários, paginação nas listagens, geração de relatórios em PDF e testes automatizados com xUnit.

9. Referências

References

- [1] MICROSOFT. *Entity Framework Core documentation*. Disponível em: <https://learn.microsoft.com/ef/core/>. Acesso em: jun. 2026.
- [2] MICROSOFT. *ASP.NET Core documentation*. Disponível em: <https://learn.microsoft.com/aspnet/core/>. Acesso em: jun. 2026.
- [3] SMARTBEAR. *Swagger / OpenAPI Specification*. Disponível em: <https://swagger.io/specification/>. Acesso em: jun. 2026.
- [4] FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese (Doutorado) — University of California, Irvine, 2000.

10. Apêndices

10.1. Repositório e Vídeo

- Repositório GitHub: <https://github.com/SEU-USUARIO/locadora-veiculos>
- Vídeo de apresentação (pitch): <https://youtube.com/SEU-VIDEO>

10.2. Lista de Endpoints

A documentação completa e interativa de todos os *endpoints* está disponível na interface Swagger, acessível em <http://localhost:5000/swagger> quando a aplicação está em execução.